In Today's task you will

- Implement Linear (also called Dense, Fully-Connected) layer as a Perceptron.
- Allow your solution to stack multiple layers to form MLP network.
- Perform forward propagation through your network.

This template implementation is similar to Pytorch framework.

Task 1a:

Declare a simple perceptron (Linear layer) that inherits defined class Module - it is here, to help you store all network layers.

The simple perceptron should be constructed of:

1. Input features
2. Followed by 1 Linear Layer with "single neuron"
3. Activation function
4. Perform forward pass for the example feature vectors `xInput1` and `xInput2` of `size = 10` features. Use prepared plot to view the results. (Repeat the process using all 4 activation functions.)

In [2]:
```
# Import
import numpy as np
from plotly.subplots import make_subplots
import plotly.graph_objects as go
from collections import OrderedDict
```

Module

All deep learning frameworks have usually one elementary building block. In our project, we follow the structure of the pytorch, so the elementary building block is called **`Module`**. Now, it is pretty simple, but it will get more complex and more useful... You can see function `.backward` that will later contain the partial derivations of chain rule for backward pass and parameter optimization.

In [3]:
```
class Module:
    def __init__(self):
        self.modules = OrderedDict()

    def add_module(self, module, name:str):
        if hasattr(self, name) and name not in self.modules:
```

```
            raise KeyError("attribute '{}' already exists".format(name))
        elif '.' in name:
            raise KeyError("module name can't contain \".\"")
        elif name == '':
            raise KeyError("module name can't be empty string \"\"")
        self.modules[name] = module

    def forward(self, *args, **kwargs) -> np.ndarray:
        pass

    def backward(self, *args, **kwargs):
        pass

    def __call__(self, *args, **kwargs):
        return self.forward(*args, **kwargs)
```

Linear Layer

In the lecture, we talked about a Perceptron and Single Layer Perceptron as an object with weight for every input value. In the frameworks, the "Fully connected layer" is implemented in Matrix Algebra.

Also, the activation function and layer logic are separated for easier backward propagation (chain rule) and optimization (The topic of 2nd+3rd lecture).

(If you want to know more, you can go to the lecture, or you can take a look on the implementation of forward and backward propagation on your own.)

In [4]:
```
#-------------------------------------------------------------------------------
#   Linear class
#-------------------------------------------------------------------------------
class Linear(Module):
    def __init__(self, in_features, out_features):
        super(Linear, self).__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.W = np.random.randn(out_features, in_features)
        self.b = np.zeros((out_features, 1))

    def forward(self, input: np.ndarray) -> np.ndarray:
        # >>>>>>>>> add here
        return np.matmul(self.W, input) + self.b
        # <<<<<<<<<

    def backward(self, dz):
        pass
```

Activations

The definitions for Sigmoid, Tanh, ReLU, and LeakyReLU activation functions with forward and backward pass. Implement the forward pass. (for now, you leave the backward pass on `pass`)

In [5]:
```
#-------------------------------------------------------------------------
#   SigmoidActivationFunction class
#-------------------------------------------------------------------------
class Sigmoid(Module):
    def __init__(self):
        super(Sigmoid, self).__init__()

    def forward(self, input: np.ndarray) -> np.ndarray:
        # >>>>>>>>> add here
        return 1.0 / (1.0 + np.exp(-input))
        # <<<<<<<<<

    def backward(self, da):
        pass

#-------------------------------------------------------------------------
#   HyperbolicTangentActivationFunction class
#-------------------------------------------------------------------------
class Tanh(Module):
    def __init__(self):
        super(Tanh, self).__init__()

    def forward(self, input: np.ndarray) -> np.ndarray:
        # >>>>>>>>> add here
        return (np.exp(input) - np.exp(-input)) / (np.exp(input) + np.exp(-i
        # <<<<<<<<<

    def backward(self, da):
        pass

#-------------------------------------------------------------------------
#   RELUActivationFunction class
#-------------------------------------------------------------------------
class ReLU(Module):
    def __init__(self):
        super(ReLU, self).__init__()

    def forward(self, input: np.ndarray) -> np.ndarray:
        # >>>>>>>>> add here
        return np.maximum(0, input)
        # <<<<<<<<<

    def backward(self, da):
        pass

#-------------------------------------------------------------------------
#   LeakyRELUActivationFunction class
#-------------------------------------------------------------------------
class LeakyReLU(Module):
    # >>>>>>>>> add something here
    def __init__(self, alpha=0.01):
```

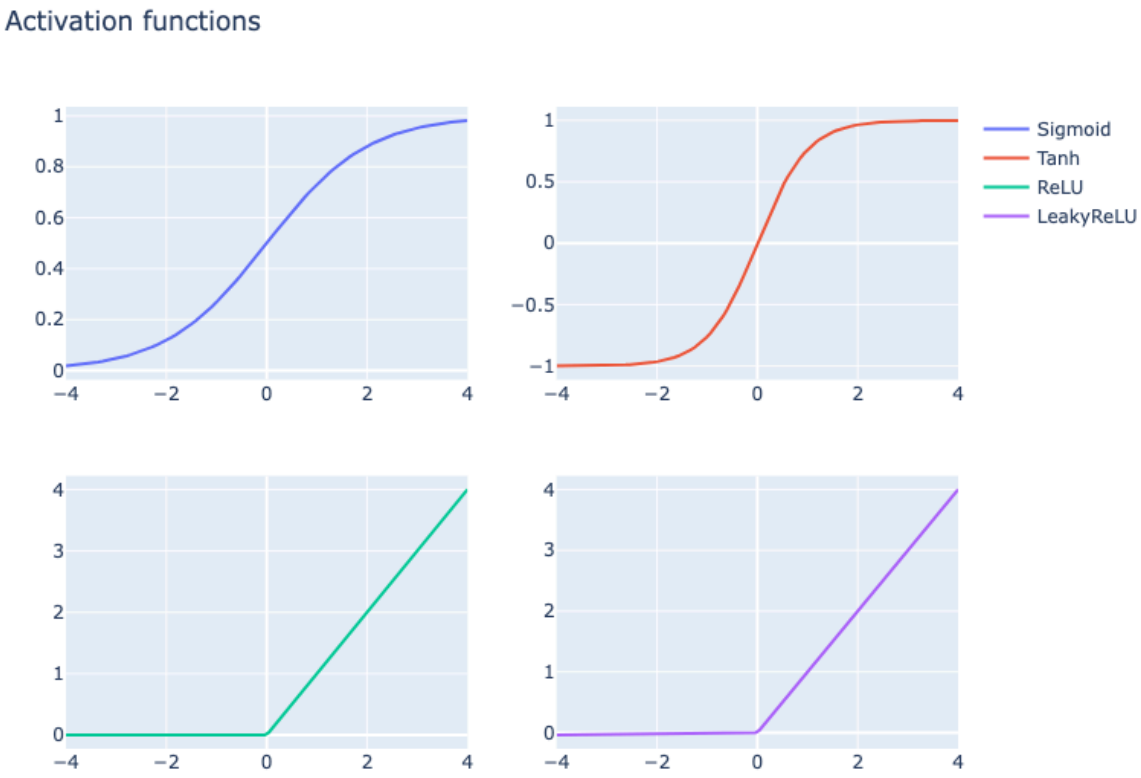
```
        super(LeakyReLU, self).__init__()
        self.alpha = alpha

    def forward(self, input: np.ndarray) -> np.ndarray:
        return np.where(input > 0, input, self.alpha * input)
    # <<<<<<<<<<<
    def backward(self, da):
        pass
```

Plotting the functions

Verify your implementations of Activation functions - do your graphs look like they should?

In [6]:

```
activationsInput = np.linspace(-4,4,100)

sigmoid = Sigmoid()
y = sigmoid.forward(activationsInput)

fig = make_subplots(rows=2, cols=2)

fig.add_trace(
    go.Scatter(x=activationsInput, y=y, name='Sigmoid'),
    row=1, col=1
)

tanh = Tanh()
y = tanh.forward(activationsInput)
fig.add_trace(
    go.Scatter(x=activationsInput, y=y, name='Tanh'),
    row=1, col=2
)

relu = ReLU()
y = relu(activationsInput)
fig.add_trace(
    go.Scatter(x=activationsInput, y=y, name='ReLU'),
    row=2, col=1
)

leakyrelu = LeakyReLU()
y = leakyrelu(activationsInput)
fig.add_trace(
    go.Scatter(x=activationsInput, y=y, name='LeakyReLU'),
    row=2, col=2
)

fig.update_layout(height=600, width=800, title_text="Activation functions")
fig.show()
```



Perceptron feed forward

Model your Perceptron. Define and initialize perceptron with "1 neuron"! Feed `xInput1` and `xInput2` to the perceptron and print the results.

In [7]:
```
# xInput1 is just a single sample - it contains 1 sample with 10 features
xInput1 = np.expand_dims(np.arange(10), axis=1)     # shape <10; 1>

# xInput2 is just a mini-batch! - it contains 4 samples with 10 features
xInput2 = np.random.randn(10, 4)                    # shape <10; 4>

# >>>>>>>>> Initialize Your Perceptron Here
perceptron = Linear(10, 1)

out1 = perceptron(xInput1)
print("xInput1 output:", out1)

out2 = perceptron(xInput2)
print("xInput2 output:", out2)
# <<<<<<<<< Use as many lines as you need
```

```
xInput1 output: [[-34.61172983]]
xInput2 output: [[ 0.89650466  3.69927168  1.53244418 -0.50570122]]
```

Your Perceptron with an Activation function. Use previously defined perceptron and use its output as input for the activation function sigmoid and LeakyReLU. Feed `xInput1`

and `xInput2` to the perceptron, print and observe the results.

In [8]:
```
# >>>>>>>>> Initialize activations and feed them after perceptron
sig = Sigmoid()
lrelu = LeakyReLU()

# xInput1
z1 = perceptron(xInput1)
print("Sigmoid(xInput1):", sig(z1))
print("LeakyReLU(xInput1):", lrelu(z1))

# xInput2
z2 = perceptron(xInput2)
print("Sigmoid(xInput2):", sig(z2))
print("LeakyReLU(xInput2):", lrelu(z2))
# <<<<<<<<< Use as many lines as you need
```

```
Sigmoid(xInput1): [[9.29644118e-16]]
LeakyReLU(xInput1): [[-0.3461173]]
Sigmoid(xInput2): [[0.71023068 0.97585582 0.82236365 0.3762018 ]]
LeakyReLU(xInput2): [[ 0.89650466  3.69927168  1.53244418 -0.00505701]]
```

Task 1b:

Finish the implementation of class `Model` - finish the call of forward feed. Declare a simple model consisting of:

1. 2 Linear Layers with arbitrary number of neurons
2. Output Linear Layer with 1 neuron.

...and activation functions to add non-linearity

Declare your own input vector with 16 features. Perform forward pass through the network and print the results.

Model class

Implementation of the **`Model`** class. Define its forward function - the implementation of forward and backward pass is sensitive to the order of called operations. Each Layer(module) of type **`Module`** can be saved to the attribute **`Module.modules`** using the **`add_module`** method.

In [9]:
```
#-------------------------------------------------------------------------
#   Model class
#-------------------------------------------------------------------------
class Model(Module):
    def __init__(self):
        super(Model, self).__init__()

    def forward(self, input):
        # >>>>>>>>> add here
```

```
        x = input
        for name, module in self.modules.items():
            x = module(x)
        return x
        # <<<<<<<< do something beautiful... and simple

    def backward(self, dA: np.ndarray):
        pass
```

In [10]:

```
model = Model()
# >>>>>>>>> Build the model architecture with 2 hidden layers and one final
model.add_module(Linear(16, 32), "linear1")
model.add_module(ReLU(), "relu1")
model.add_module(Linear(32, 16), "linear2")
model.add_module(ReLU(), "relu2")
model.add_module(Linear(16, 1), "output")
model.add_module(Sigmoid(), "sigmoid_out")

# input with 16 features, 1 sample
x = np.random.randn(16, 1)
result = model(x)
print("Model output:", result)
```

```
Model output: [[1.19051243e-16]]
```